Исследование поиска в ширину

Цель работы: приобретение навыков по работе с поиском в ширину с помощью языка программирования Python версии 3.x

```
(venv) PS C:\!lab2\lab2> python main.py
============================================================
Лабораторная работа: Исследование поиска в ширину (BFS)
============================================================

Выберите задание:
1. Пример BFS на простом графе (с визуализацией)
2. Задача: Подсчет островов в матрице
3. Задача: Кратчайший путь в лабиринте
4. Задача: Поиск пути в графе Румынии (с визуализацией)
5. Сравнение с ручным решением
6. Задача: Льющиеся кувшины (реализация примера)
0. Выход

Ваш выбор: 1
Пример BFS на простом графе:
Граф: A -> B, C; B -> D, E; C -> F; E -> F

Найден путь: A -> C -> F
Длина пути: 2 шагов

Визуализация графа с найденным путем...
```

Рисунок 1 - Пример метода исследования поиска в ширину (BFS) на простом графе

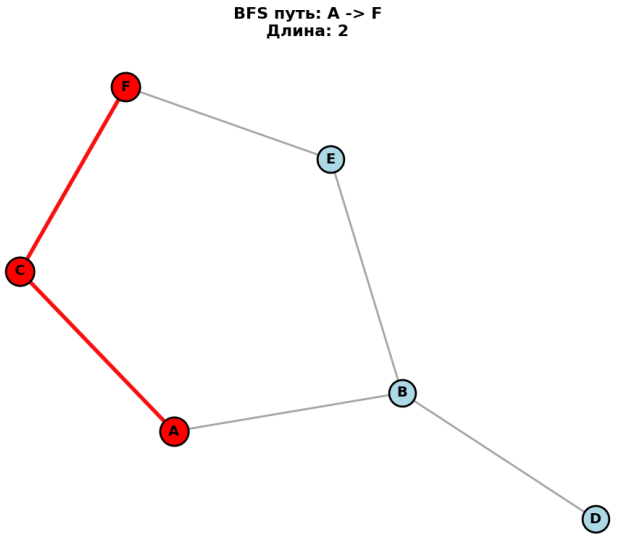

Рисунок 2 - Визуализация графа примера метода исследования поиска в ширину (BFS) на простом графе

```
Ваш выбор: 2
============================================================
Задача: Расширенный подсчет количества островов в бинарной матрице
0 - вода, 1 - земля
Острова могут соединяться по вертикали, горизонтали и диагонали
============================================================

Матрица 5x5:
  0 0 0 1 1
  0 1 0 0 1
  1 0 0 1 1
  0 0 0 0 0
  1 0 1 0 1

Общее количество островов: 5

Детализация островов:
  Остров 1: (0,3), (0,4), (1,4), (2,3), (2,4)
  Остров 2: (1,1), (2,0)
  Остров 3: (4,0)
  Остров 4: (4,2)
  Остров 5: (4,4)

============================================================
Итог: в матрице обнаружено 5 островов
============================================================
```

Рисунок 3 – Расширенный подсчет количества островов в бинарной матрице

```
Ваш выбор: 3
============================================================
Задача: Поиск кратчайшего пути в лабиринте
1 - проход, 0 - стена
============================================================

Лабиринт (8x8):
  1 1 0 1 1 1 0 1
  1 0 1 1 0 1 1 1
  1 1 1 0 1 1 0 1
  0 0 1 1 1 0 1 1
  1 1 1 0 1 1 1 0
  0 1 0 1 0 1 0 1
  1 1 1 1 0 1 1 1
  1 0 0 1 1 1 0 1

Начальная точка: (0, 0)
Конечная точка: (7, 7)

Длина кратчайшего пути: 14

Визуализация пути:
S - старт, E - конец, P - путь, █ - стена, · - проход
  S · █ · · · █ ·
  P █ · · █ · · ·
  P P P █ · · █ ·
  █ █ P P P █ · ·
  · · · █ P P · █
  █ · █ · █ P █ ·
  · · · · █ P P P
  · █ █ · · · █ E

Последовательность координат пути (15 точек):
  Шаг  0: (0, 0)
  Шаг  1: (1, 0)
  Шаг  2: (2, 0)
  Шаг  3: (2, 1)
  Шаг  4: (2, 2)
  Шаг  5: (3, 2)
  Шаг  6: (3, 3)
  Шаг  7: (3, 4)
  Шаг  8: (4, 4)
  Шаг  9: (4, 5)
  Шаг 10: (5, 5)
  Шаг 11: (6, 5)
  Шаг 12: (6, 6)
  Шаг 13: (6, 7)
  Шаг 14: (7, 7)

============================================================
Итог: длина кратчайшего пути = 14
============================================================
```

Рисунок 4 - Поиск кратчайшего пути в лабиринте

```
Ваш выбор: 4
Путь из Arad в Bucharest:
Arad -> Sibiu -> Fagaras -> Bucharest
Длина пути (в шагах): 3
Общая стоимость: 450

Визуализация графа с найденным путем...
```

Рисунок 5 - программа для графа из лабораторной работы 1

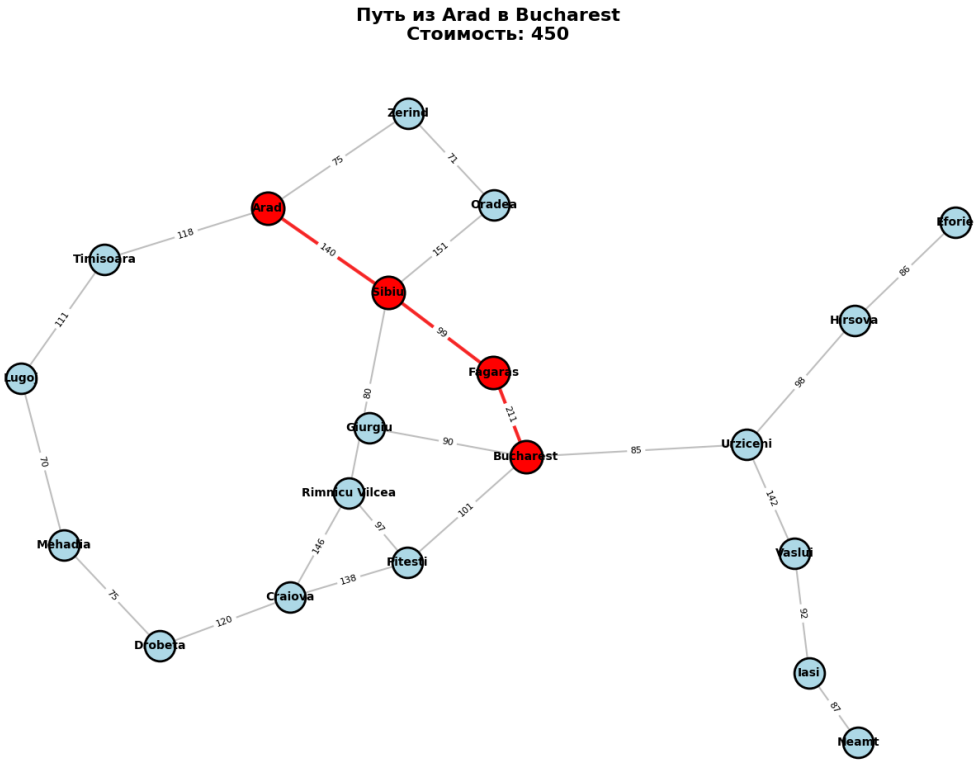

Рисунок 6 – Визуализация графа минимального расстояния между начальным и конечным пунктами

```
Ваш выбор: 5

=== Сравнение результатов ===
Ручное решение (из первой лабы) для Arad -> Bucharest:
Путь: Arad -> Sibiu -> Fagaras -> Bucharest
Стоимость: 140 + 99 + 211 = 450

Решение BFS (поиск в ширину):
Путь из Arad в Bucharest:
Arad -> Sibiu -> Fagaras -> Bucharest
Длина пути (в шагах): 3
Общая стоимость: 450

BFS нашел путь: Arad -> Sibiu -> Fagaras -> Bucharest
Стоимость BFS пути: 450

Вывод: BFS находит путь с минимальным количеством шагов,
но не обязательно с минимальной стоимостью!
Для минимальной стоимости нужен алгоритм A* или Дейкстры.
```

Рисунок 7 - Сравнение результатов поиска в ширину и ручным решением

```
Ваш выбор: 6
============================================================
Задача о льющихся кувшинах
============================================================
Начальное состояние: (1, 1, 1)
Целевой объем: 13
Емкости кувшинов: (2, 16, 32)

Последовательность действий:
[('Fill', 1), ('Pour', 1, 0), ('Dump', 0), ('Pour', 1, 0)]

Последовательность состояний:
[(1, 1, 1), (1, 16, 1), (2, 15, 1), (0, 15, 1), (2, 13, 1)]

Итог: достигнут объем 13 в кувшине 1
```

Рисунок 8 - Реализация задачи о льющихся кувшинах

Ответы на вопросы:

1. FIFO очередь (первый пришел — первый ушел).
2. Чтобы сохранить порядок обхода по уровням, сначала обрабатываем все узлы текущего уровня, потом переходим к следующему.
3. Они обрабатываются первыми, FIFO принцип.
4. Все дочерние узлы корневого узла в порядке их генерации.

5. Чтобы найти кратчайший путь (минимальное количество шагов) в невзвешенном графе.

6. O(b^d) — где b — коэффициент разветвления, d — глубина решения.

7. Размер очереди frontier, хранит все узлы текущего и следующего уровней.

8. При конечном коэффициенте разветвления и положительных стоимостях шагов.

9. Экспоненциальный рост памяти, хранит все узлы текущего уровня.

10. Находит оптимальное решение, кратчайший путь по количеству шагов.

11. Находит путь от начального состояния к целевому с использованием метода поиска в ширину.

12. Объект задачи определяет начальное состояние, цель и переходы.

13. Создаёт начальный узел как точка старта поиска.

14. Возвращается начальный узел так как решение уже найдено.

15. FIFOQueue — обеспечивает порядок обхода по уровням.

16. Отслеживает посещённые состояния, предотвращает повторное посещение.

17. Избежать циклов и повторной работы это экономит время и память.

18. Основной цикл поиска, продолжается пока есть узлы для обработки.

19. Извлекается для расширения — проверяется, и генерируются потомки.

20. Генерирует все возможные следующие состояния из текущего узла.

21. problem.is_goal(s) - проверка равенства состояния целевому

22. Добавляется в очередь для дальнейшего исследования.

23. Ошибка вопроса - в BFS дочерние узлы добавляются в КОНЕЦ очереди (append, не appendleft).

24. Специальный узел failure - указывает на отсутствие решения.

25. Узел-индикатор неудачи - возвращается когда цель недостижима.

Вывод: в процессе выполнения лабораторной работы были приобретены навыков по работе с поиском в ширину с помощью языка программирования Python версии 3.x.